

ЛР19 (3 пары / 6 часов)

ЛР19 (3 пары / 6 часов)

«Роли и доступ: User/Admin. Запрет опасных команд»

0) Итог ЛР19 (что должно быть готово)

В config.json есть поле Role (User или Admin)

При запуске приложение показывает роль

Ограничения доступа:

User не может удалять (4)

User не может импортировать (11)

User не может менять настройки (15)

При запрете выводится сообщение и пишется лог

Есть отчёт docs/LR19_Report.md

Минимум 3 коммита

1) Теория (важное)

1.1. Роли

Роли нужны для разделения прав:

обычный пользователь имеет доступ только к безопасным функциям
администратор может выполнять опасные операции

1.2. Принцип минимальных прав

По умолчанию пользователь получает минимальный набор возможностей.

2) Практика. Пара 1 (2 часа) — добавление Role в config

Шаг 1. Добавить Role в AppConfig

Открыть AppConfig.cs и добавить:

```
public string Role { get; set; } = "User";
```

Итоговая структура должна содержать поле Role.

Шаг 2. Сохранить config.json с новым полем

После загрузки cfg можно вызвать:

```
configService.Save(cfg);
```

Шаг 3. Показать роль при запуске

В Program.cs добавить вывод:

```
Console.WriteLine($"Role: {cfg.Role}");
```

Шаг 4. Коммит №1

Сообщение:

```
feat: add Role to config
```

Commit → Push.

3) Практика. Пара 2 (2 часа) — AccessControl и запреты

Шаг 5. Создать папку Security

В TaskTracker.App создать папку:

Security

Шаг 6. Создать AccessControl.cs

Создать файл TaskTracker.App → Security → AccessControl.cs:

```
namespace TaskTracker.App.Security;
```

```
public static class AccessControl
{
    public static bool IsAdmin(string? role)
    {
        return string.Equals(role, "Admin", StringComparison.OrdinalIgnoreCase);
    }

    public static void RequireAdmin(string? role)
    {
        if (!IsAdmin(role))
            throw new ArgumentException("Недостаточно прав. Нужно: Admin");
    }
}
```

Шаг 7. Подключить using в Program.cs

Добавить:

```
using TaskTracker.App.Security;
```

Шаг 8. Ограничить команды Delete, Import, Settings

В обработке команд добавить проверку:

Пример для Delete (4):

```
if (input == "4")
{
    SafeRunner.Run("DELETE", logger, () ⇒
```

```

{
    AccessControl.RequireAdmin(cfg.Role);
    MenuHandlers.DeleteTask(service, storage, logger);
});

continue;
}

```

Пример для Import (11):

```

if (input == "11")
{
    SafeRunner.Run("IMPORT", logger, () ⇒
    {
        AccessControl.RequireAdmin(cfg.Role);
        // существующий код импорта
    });

    continue;
}

```

Пример для Settings (15):

```

if (input == "15")
{
    SafeRunner.Run("SETTINGS", logger, () ⇒
    {
        AccessControl.RequireAdmin(cfg.Role);
        // существующий код настроек
    });

    continue;
}

```

Шаг 9. Коммит №2

Сообщение:

feat: add access control and restrict commands

Commit → Push.

4) Практика. Пара 3 (2 часа) — проверка и отчёт

Шаг 10. Проверка роли User

В config.json поставить:

```
"Role": "User"
```

Запустить приложение и проверить:

Delete (4) → отказ

Import (11) → отказ

Settings (15) → отказ

Шаг 11. Проверка роли Admin

Поставить:

```
"Role": "Admin"
```

Запустить приложение и проверить:

Delete (4) работает

Import (11) работает

Settings (15) работает

Шаг 12. Отчёт docs/LR19_Report.md

Создать файл:

Отчёт ЛР19 — роли и доступ## Что сделано

- В config.json добавлено поле Role (User/Admin)
- Реализован AccessControl

- Ограничены команды:

- Delete (4)
- Import (11)
- Settings (15)

Как проверить

1) Role=User → Delete/Import/Settings недоступны

2) Role=Admin → все команды доступны

Шаг 13. Коммит №3

Сообщение:

docs: add LR19 report

Commit → Push.

5) Что сдавать по ЛР19

Role в config.json

ограничения на Delete/Import/Settings

docs/LR19_Report.md

минимум 3 коммита